

Lecture 1

Introduction to Software Engineering

Software Engineering

DR. NDUDI EZEAMUZIE
SCHOOL OF SCIENCE AND TECHNOLOGY



Lecture 1

Learning Outcomes

- ❑ Explain what software engineering entails.
- ❑ Describe the importance of software engineering
- ❑ Discuss the ethical issues and considerations in software engineering



Courtesy: Materials are modified from "Software Engineering", 10th Edition. by Ian Sommerville

Professional Software Development

Software Engineering

- ❑ an **engineering discipline** concerned with **all aspects of software production**
 - from the early stages of system specification to maintaining the system after it has gone into use
- ❑ **Engineering discipline**
 - connotes use of appropriate ***theories, methods and tools*** for professional software development.
- ❑ **All aspects of software production** →
 - Not just the technical process of development.
 - Includes project management and the development of tools and methods to support software production.

Why Software Engineering - I

- ✧ The economies of ALL developed nations are dependent on software.
- ✧ Expenditure on software represents a significant fraction of Gross National Product (GNP) in all developed countries (even developing countries!).
- ✧ More and more systems are software controlled

Therefore, we need to be able to produce **reliable** and **trustworthy** systems **economically** and **quickly**.

Why Software Engineering - II

- ✧ Software are abstract and intangible
- ✧ Usually not constrained by materials and regulations
- ✧ Potential to become extremely complex
- ✧ Software are Costly to develop and maintain
 - The costs of **software** on a PC are often greater than the hardware cost.

Therefore, we need **cost-effective** ways for managing software development.

Why Software Projects Fail

1. Increasing system complexity

- ✧ Software needs to be built and delivered more quickly
- ✧ Systems have new capabilities that were previously thought to be impossible.
- ✧ New software engineering techniques are required to deliver complex systems

2. Failure to use software engineering methods

- ✧ It is easy to write computer programs without using software engineering methods and techniques.
- ✧ Such approach makes software more expensive and less reliable than it should be.

Software and Software Engineering - I

Software	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
Attributes of good software	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
Software engineering?	Software engineering is an engineering discipline that is concerned with all aspects of software production.
Fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
Difference between software engineering and computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.

Software and Software Engineering - II

Key challenges facing software engineering	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
Costs of software engineering	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
Best software engineering techniques and methods	It depends 😊. Different techniques are appropriate for different types of system. For instance, game developers may use prototypes . Conversely, critical control systems require a complete and analyzable specification to be developed.
Impact of web on software engineering	The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Software Product - I

Broadly, software may be classified into two

1. Generic products

- ❑ Stand-alone systems sold to open market.
- ❑ Examples –graphics programs, CAD software; appointment systems

2. Customized products

- ❑ Commissioned by and developed for a specific customer
- ❑ Often to meet customers' personal or business needs.
- ❑ Examples – embedded control systems, air traffic control software, and traffic monitoring systems.

Software Product - II

Major Difference between Generic and Customized Products:

- ❑ Who controls the software specification?
- ❑ Distinction is increasing blurry →
 - Some generic software may be a base for customization
 - Examples – Enterprise Resource Planning (ERP) systems

Attributes of Good Software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers . This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software Process Activities

- ❑ Software Process → A systematic approach to software engineering
- ❑ Four key activities
 - ❑ **Software specification**
 - Define what features are to be produced in the software and the constraints on its operation.
 - ❑ **Software development**
 - Entails designing and programming
 - ❑ **Software validation**
 - Check to ensure that software meets customer requirements
 - ❑ **Software evolution**
 - Modifying software to reflect changing customer and market requirements.

General Issues Affecting Software

❑ Heterogeneity

- operating as distributed systems across networks, interoperating among different types of devices/OS, and cope with legacy system in other languages.

❑ Business and Social Change

- Business and societal needs are changing rapidly, and new technologies are emerging too.

❑ Security and Trust

- As software is intertwined with all aspects of our lives, it is essential that we can trust that software.

❑ Scale

- Software vary across wide range of scales; from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

Software Engineering Diversity

- ❑ There is **no universal** set of software engineering techniques that is applicable to all types of software
 - Recall, there are different types of software systems
 - Examples: stand-alone applications, interactive transaction-based (web) applications, embedded control systems, batch processing systems, entertainment system, modelling/simulation systems, data collection/analysis system etc.
- ❑ Often ,software engineering methods and tools will depend on
 - the type of application being developed,
 - the requirements of the customer, and
 - the background of the development team.
- ❑ However, there are several **software engineering fundamentals** apply in most systems.

Software Engineering Fundamentals

There are several software engineering fundamentals apply in most systems.

Irrespective of the development technique.

- ❑ Use **managed** and **understood** development process.
- ❑ Consider dependability and performance
- ❑ Understand and manage the software **specification** and **requirements**.
- ❑ Where appropriate, **reuse existing resources / software** rather than write new software.

Software Engineering : Internet/Web-based System

Important developments in internet/web software engineering

- ❑ Organizations are increasingly developing web-based systems rather than local systems.
- ❑ The fundamental ideas of software engineering apply to web-based software
- ❑ *Software reuse* is the dominant approach for constructing web-based systems. Think about how you can assemble software from pre-existing software components and systems.
- ❑ *Incremental and agile* development process is recommended.
 - Often, it is impractical to specify all the requirements for software
- ❑ **Service-oriented** software engineering, where the software components are stand-alone web services.
- ❑ **Rich interfaces** are supported by technologies such as AJAX and HTML5

Software Engineering Ethics

Software Engineering Ethics

- ❑ Software engineering involves wider responsibilities
 - Just the application of technical skills is **not enough**
 - Like other engineering professions, there are social dimensions and legal frameworks
- ❑ Professional software engineers have obligation to ethical and moral responsibilities
 - More than simply upholding the laws
 - Honesty and Integrity
 - Avoid actions and activities that may bring disrepute to the profession

Professional Responsibility

❑ Confidentiality

- Towards employers and clients

❑ Competence

- Don't misrepresent or accept works outside your competence

❑ Intellectual Property Right

- Be aware of local laws such as patent and copyright

❑ Computer Misuse

- Your technical skills should not be used to infringe on other people's computer

Roles of Professional Societies in Ethics

Association for Computing Machinery (ACM) / Institute of Electrical and Electronics Engineers (IEEE)

Software Engineering Code of Ethics and Professional Practices

- ❑ Members sign up to the code of practice when they join the organization/society
- ❑ The code contains **eight** principles related to the behaviour of and decisions made by professional software engineers.
- ❑ Rationale for Code of Ethics
 - Computers (Software) have a central and growing role in every sector of the society
 - Software engineers have significant opportunities to do good or cause harm
 - Ensure that software engineers commit to good use of their knowledge and skills.

The ACM/IEEE Code of Ethics – I

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

The ACM/IEEE Code of Ethics – II

1. **PUBLIC** - Software engineers shall act consistently with the public interest.
2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. **PRODUCT** - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. **JUDGMENT** - Software engineers shall maintain integrity and independence in their professional judgment.
5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. **PROFESSION** - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. **COLLEAGUES** - Software engineers shall be fair to and supportive of their colleagues.
8. **SELF** - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Summary - I

- ❑ Software engineering is an engineering discipline that is concerned with all aspects of software production.
- ❑ Essential software product attributes are maintainability, dependability and security, efficiency and acceptability.
- ❑ The high-level activities of specification, development, validation and evolution are part of all software processes.
- ❑ The fundamental notions of software engineering are universally applicable to all types of system development.

Summary - II

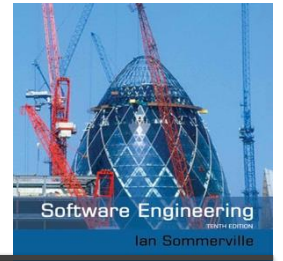
- ❑ There are many different types of system and each requires appropriate software engineering tools and techniques for their development.
- ❑ The fundamental ideas of software engineering are applicable to all types of software system.
- ❑ Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.
- ❑ Professional societies publish codes of conduct which set out the standards of behaviour expected of their members.



Thanks for
listening 😊

Case studies (Optional)

Case studies



✧ 1. A personal insulin pump

- An embedded system in an insulin pump used by diabetics to maintain blood glucose control.

✧ 2. A mental health case patient management system

- Mentcare. A system used to maintain records of people receiving care for mental health problems.

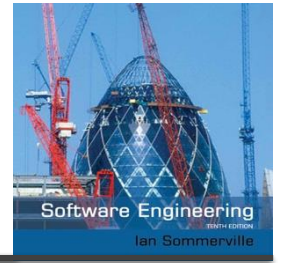
✧ 3. A wilderness weather station

- A data collection system that collects data about weather conditions in remote areas.

✧ 4. iLearn: a digital learning environment

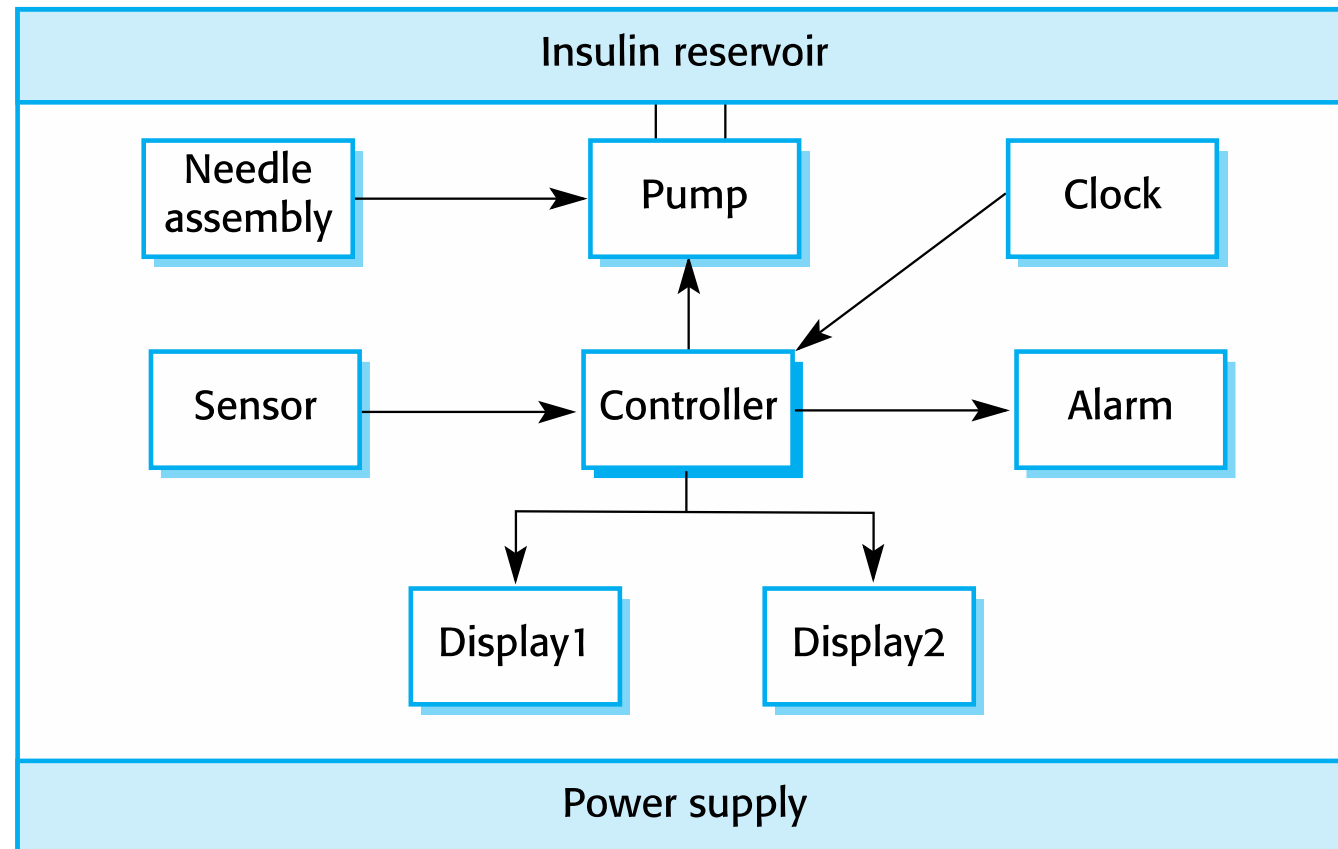
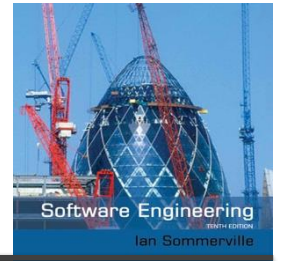
- A system to support learning in schools

Insulin pump control system

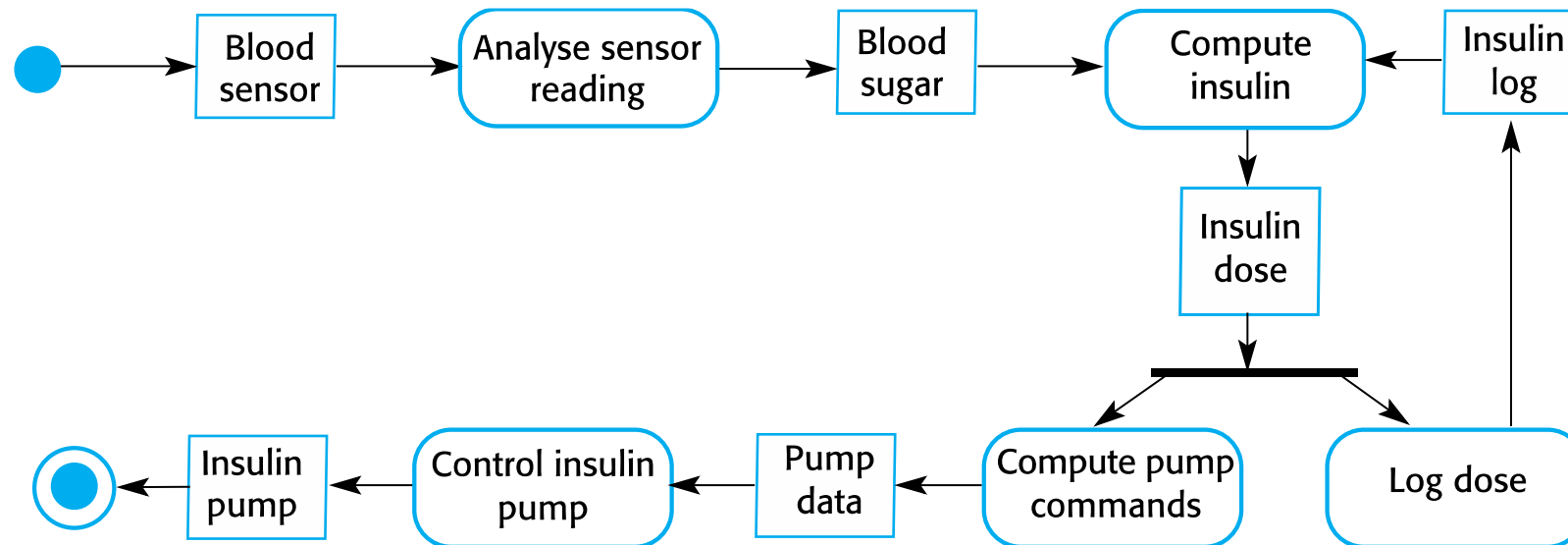
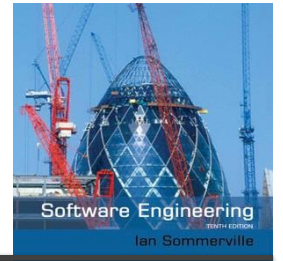


- ✧ Collects data from a blood sugar sensor and calculates the amount of insulin (胰岛素) required to be injected.
- ✧ Calculation based on the rate of change of blood sugar levels.
- ✧ Sends signals to a micro-pump to deliver the correct dose of insulin.
- ✧ **Safety-critical system** as low blood sugars can lead to brain malfunctioning, coma (昏迷) and death; high-blood sugar levels have long-term consequences such as eye and kidney damage.

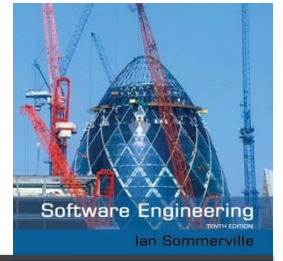
Insulin pump hardware architecture



Activity model of the insulin pump

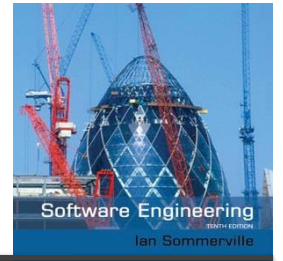


Essential high-level requirements

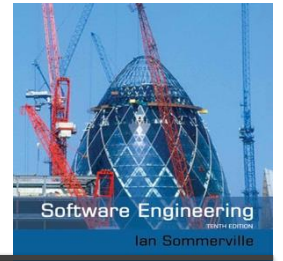


- ✧ The system shall be available to deliver insulin when required.
- ✧ The system shall perform **reliably** and deliver the correct amount of insulin to counteract the current level of blood sugar.
- ✧ The system must therefore be designed and implemented to ensure that the system always meets these requirements.

Mentcare: A patient information system for mental health care

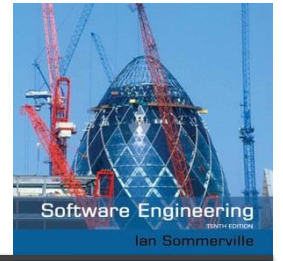


- ✧ ***A patient information system*** to support mental health care is a medical information system that maintains information about patients suffering from mental health problems and the treatments that they have received.
- ✧ Most mental health patients do not require dedicated hospital treatment but need to attend specialist clinics regularly where they can meet a doctor who has detailed knowledge of their problems.
- ✧ To make it easier for patients to attend, these clinics are not just run in hospitals. They may also be held in local medical practices or community centres.



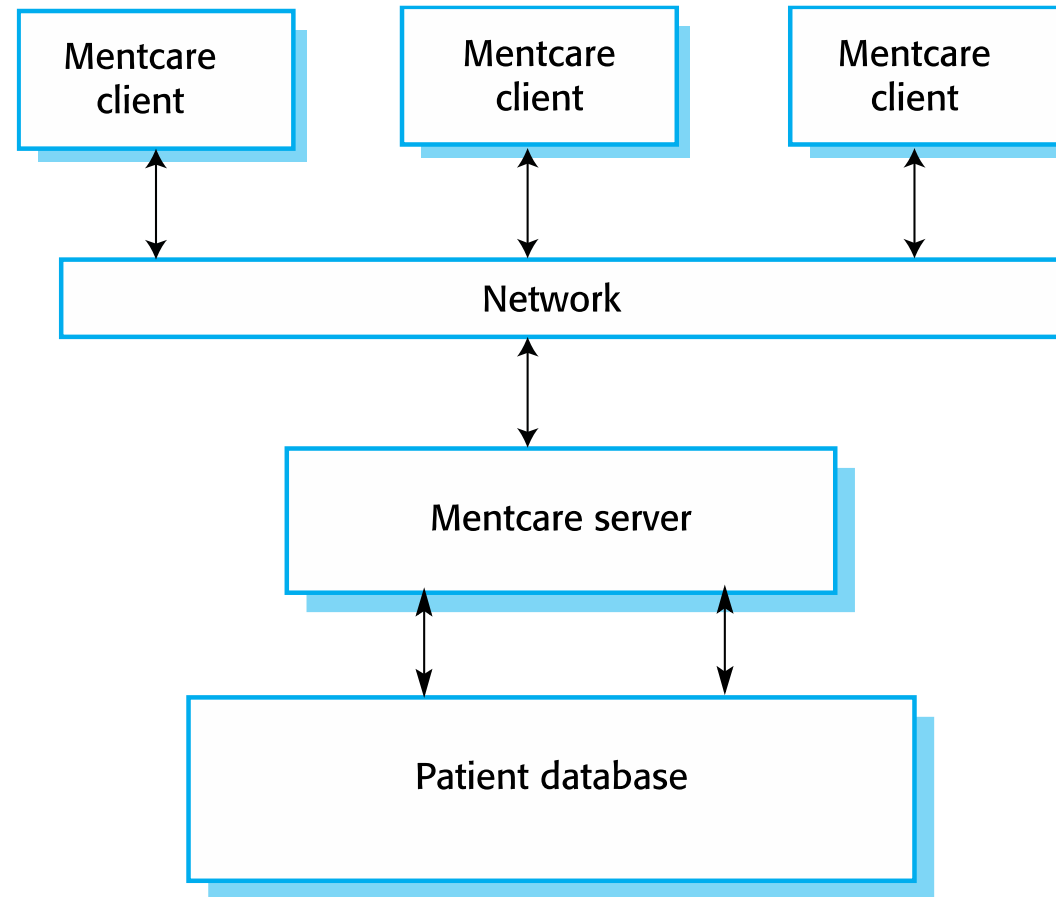
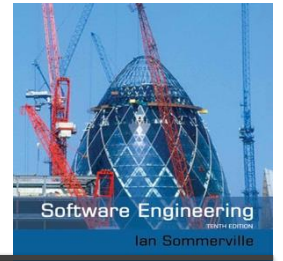
- ✧ Mentcare is an ***information system*** that is intended for use in clinics.
- ✧ It makes use of a centralized database of patient information but has also been designed to run on a PC, so that it may be accessed and used from sites that do not have secure network connectivity.
- ✧ When the local systems have secure network access, they use patient information in the database but they can download and use local copies of patient records when they are disconnected.

Mentcare goals

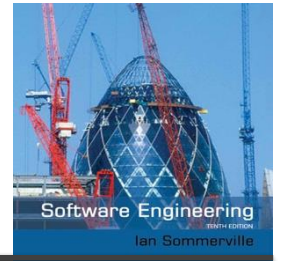


- ✧ To generate management information that allows health service managers to assess performance against local and government targets.
- ✧ To provide medical staff with timely information to support the treatment of patients.

The organization of the Mentcare system



Key features of the Mentcare system



✧ *Individual care management*

- Clinicians can create records for patients, edit the information in the system, view patient history, etc. The system supports data summaries so that doctors can quickly learn about the key problems and treatments that have been prescribed.

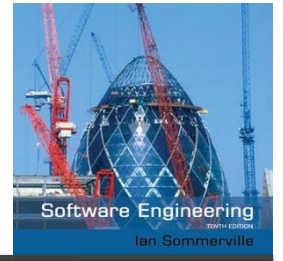
✧ *Patient monitoring*

- The system monitors the records of patients that are involved in treatment and issues warnings if possible problems are detected.

✧ *Administrative reporting*

- The system generates monthly management reports showing the number of patients treated at each clinic, the number of patients who have entered and left the care system, number of patients sectioned, the drugs prescribed and their costs, etc.

Mentcare system concerns



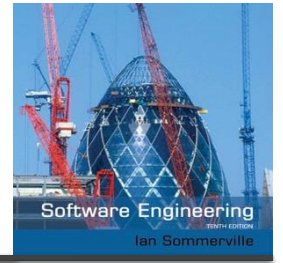
✧ **Privacy**

- It is essential that patient information is confidential and is never disclosed to anyone apart from authorized medical staff and the patient themselves.

✧ **Safety**

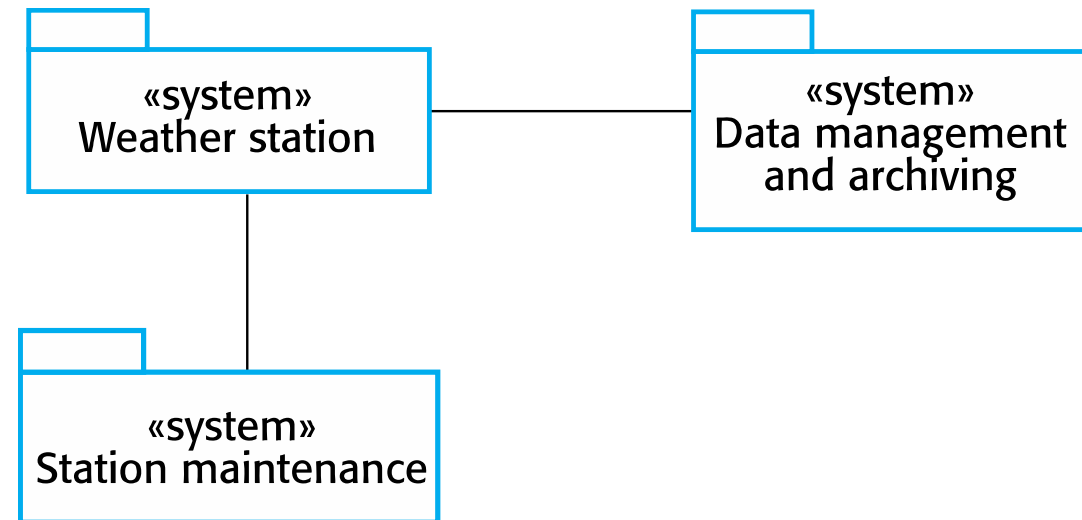
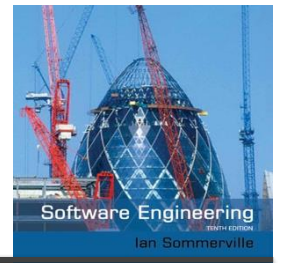
- Some mental illnesses cause patients to become suicidal or a danger to other people. Wherever possible, the system should warn medical staff about potentially suicidal or dangerous patients.
- The system must be available when needed otherwise, safety may be compromised and it may be impossible to prescribe the correct medication to patients.

Wilderness (荒野) weather station

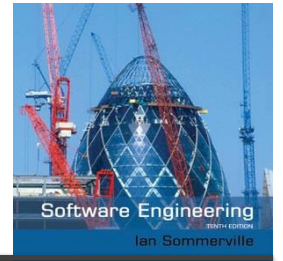


- ✧ The government of a country with large areas of wilderness decides to deploy several hundred weather stations in remote areas.
- ✧ Weather stations collect data from a set of instruments that measure temperature and pressure, sunshine, rainfall, wind speed and wind direction.
 - The weather station includes a number of instruments that measure weather parameters such as the wind speed and direction, the ground and air temperatures, the barometric pressure (大氣壓力) and the rainfall over a 24-hour period. Each of these instruments is controlled by a software system that takes parameter readings periodically and manages the data collected from the instruments.

The weather station's environment



Weather information system



✧ The weather station system

- This is responsible for collecting weather data, carrying out some initial data processing and transmitting it to the data management system.

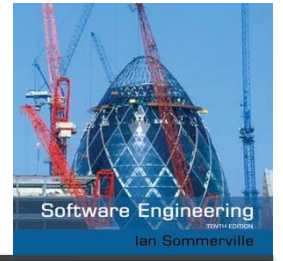
✧ The data management and archiving system

- This system collects the data from all of the wilderness weather stations, carries out data processing and analysis and archives the data.

✧ The station maintenance system

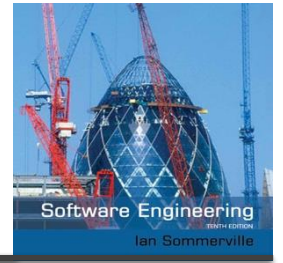
- This system can communicate by satellite with all wilderness weather stations to monitor the health of these systems and provide reports of problems.

Additional software functionality



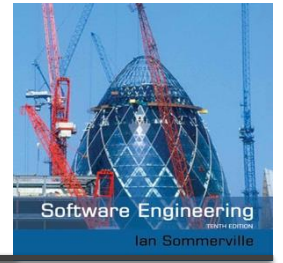
- ✧ Monitor the instruments, power and communication hardware and report faults to the management system.
- ✧ Manage the system power, ensuring that batteries are charged whenever the environmental conditions permit but also that generators are shut down in potentially damaging weather conditions, such as high wind.
- ✧ Support dynamic reconfiguration where parts of the software are replaced with new versions and where backup instruments are switched into the system in the event of system failure.

iLearn: A digital learning environment

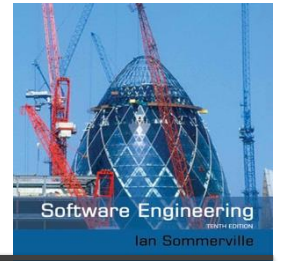


- ✧ A digital learning environment is a framework in which a set of general-purpose and specially designed tools for learning may be embedded plus a set of applications that are geared to the needs of the learners using the system.
- ✧ The tools included in each version of the environment are chosen by teachers and learners to suit their specific needs.
 - These can be general applications such as spreadsheets, learning management applications such as a Virtual Learning Environment (VLE) to manage homework submission and assessment, games and simulations.

Service-oriented systems

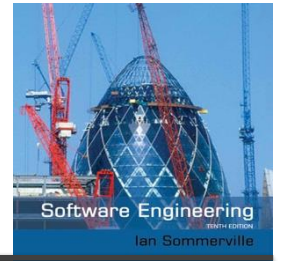


- ✧ The system is a service-oriented system with all system components considered to be a replaceable service.
- ✧ This allows the system to be updated incrementally as new services become available.
- ✧ It also makes it possible to rapidly configure the system to create versions of the environment for different groups such as very young children who cannot read, senior students, etc.



- ✧ *Utility services* that provide basic application-independent functionality and which may be used by other services in the system.
- ✧ *Application services* that provide specific applications such as email, conferencing, photo sharing etc. and access to specific educational content such as scientific films or historical resources.
- ✧ *Configuration services* that are used to adapt the environment with a specific set of application services and do define how services are shared between students, teachers and their parents.

iLearn architecture



Browser-based user interface

iLearn app

Configuration services

Group
management

Application
management

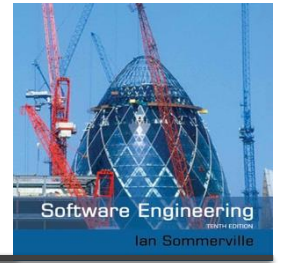
Identity
management

Application services

Email Messaging Video conferencing Newspaper archive
Word processing Simulation Video storage Resource finder
Spreadsheet Virtual learning environment History archive

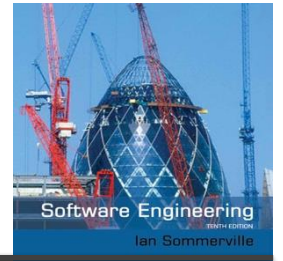
Utility services

Authentication Logging and monitoring Interfacing
User storage Application storage Search



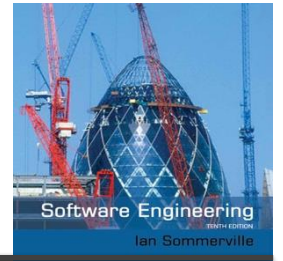
- ✧ *Integrated services* are services which offer an API (application programming interface) and which can be accessed by other services through that API. Direct service-to-service communication is therefore possible.
- ✧ *Independent services* are services which are simply accessed through a browser interface and which operate independently of other services. Information can only be shared with other services through explicit user actions such as copy and paste; re-authentication may be required for each independent service.

Key points



- ✧ Software engineering is an engineering discipline that is concerned with all aspects of software production.
- ✧ Essential software product attributes are maintainability, dependability and security, efficiency and acceptability.
- ✧ The high-level activities of specification, development, validation and evolution are part of all software processes.
- ✧ The fundamental notions of software engineering are universally applicable to all types of system development.

Key points



- ✧ There are many different types of system and each requires appropriate software engineering tools and techniques for their development.
- ✧ The fundamental ideas of software engineering are applicable to all types of software system.
- ✧ Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.
- ✧ Professional societies publish codes of conduct which set out the standards of behaviour expected of their members.